Slide 1: Title Slide

Speaker Notes:

Good morning respected panel members.

I am Apekshya Koiralal, and along with my teammates Arun Aryal and Binit Awasthi, we will briefly present our minor project titled as “A Handwritten One-Dimensional Mathematical Expression Recognition System Using CNN–RNN Architecture.”

Slide 2: Introduction

Speaker Notes:

Handwritten mathematical expression recognition focuses on converting handwritten equations into machine-readable text.

In our project, we focus specifically on one-dimensional expressions, such as linear equations written left-to-right.

The system is trained on thousands of handwritten samples and uses deep learning models to automatically recognise symbol sequences without manual intervention.

This has practical applications in digital note-taking, education platforms, and document digitisation.

Slide 3: Problem Statements

Speaker Notes:

There are several challenges in handwritten math recognition.

First, there is a lack of accurate and reliable systems for recognising handwritten mathematical expressions.

Second, handwriting varies greatly from person to person in terms of style, spacing, and symbol shapes.

Third, traditional systems depend heavily on manual symbol segmentation, which is error-prone.

Therefore, there is a need for an automated, segmentation-free, and high-accuracy recognition system.

Slide 4: Objectives

Speaker Notes:

The primary objective of our project is to develop a deep learning-based system that can accurately recognise handwritten one-dimensional mathematical expressions.

Another key objective is to study and evaluate CNN–RNN–CTC architectures, which are well-suited for sequence modelling tasks like handwriting recognition.

Slide 5: Literature Review

Speaker Notes:

Early research, such as the work by Mori and Suen, relied on rule-based and template-matching techniques, which worked only for isolated characters and required handcrafted features.

Later, Graves and Schmidhuber introduced recurrent neural networks, especially LSTMs, which significantly improved handwritten text recognition by capturing long-term dependencies.

Recent studies use end-to-end CNN–RNN–CTC architectures, which inspire the design of our proposed system.

Slide 6: System Design

Speaker Notes:

This slide shows the overall system design.

The process starts with handwritten input on a digital canvas.

The image is preprocessed and passed through the deep learning model.

Finally, the recognised expression is converted into machine-readable text and displayed to the user.

Slide 7: Proposed Block Diagram

Speaker Notes:

The block diagram explains the internal working of our system.

First, the input image undergoes preprocessing such as resizing and noise removal.

The CNN extracts visual features from the handwritten symbols.

The BiLSTM network models the sequential relationship between symbols.

Finally, CTC decoding converts the predicted sequence into the final mathematical expression without explicit segmentation.

Slide 8: Dataset

Speaker Notes:

Our dataset consists of handwritten mathematical expression images paired with their corresponding labels.

Each entry includes an image ID, file name, expression label, and image dimensions.

We plan to use a minimum of 1000 image-label pairs to ensure basic model generalisation.

Slide 9: Implementation

Speaker Notes:

The system is implemented using a CNN–RNN-based architecture.

The CNN extracts spatial features, while the Bidirectional LSTM captures context from both left-to-right and right-to-left directions.

We use CTC loss and decoding, which allows the model to align predictions with labels automatically, eliminating the need for manual symbol segmentation.

Slide 10: Tools and Techniques

Speaker Notes:

Python is used as the primary programming language.

TensorFlow and Keras are used for building and training the deep learning models.

FastAPI will serve as the backend, while HTML, CSS, and Next.js will be used for the frontend interface.

OpenCV is used for image preprocessing, and NumPy, Pandas, and Matplotlib support data handling and visualisation.

Slide 11: Expected Output

Speaker Notes:

The expected output is the conversion of handwritten mathematical expressions into textual form.

The system should accurately recognise linear expressions despite variations in handwriting style and spacing.

The output will be displayed in real time, and performance will be evaluated using character-level and sequence-level accuracy metrics.

Slide 12: Limitations

Speaker Notes:

Our system is limited to one-dimensional mathematical expressions.

The accuracy heavily depends on the quality and size of the training dataset.

Extremely unclear or irregular handwriting may reduce performance, which is a known limitation of data-driven models.

Slide 13: References

Speaker Notes:

This slide lists the key research papers and sources that guided our project design, particularly works related to CNN, RNN, and CTC-based handwriting recognition systems.

Slide 14: Any Queries?

Speaker Notes:

Thank you for your time.

We are open to any questions or suggestions from the panel.